

Escaping the AST

A Data-Oriented, Lock-Free Parallel Compiler Architecture

ADITYA KUMAR

CPPCON 2026 · 60 MIN

ESCAPING THE **AST**

Data-oriented design in the Vx compiler frontend:
flat arrays, 256-bit identities, and a pipeline with no locks to
contend for.

Aditya Kumar (x.com/adityazero_)
the Vx language project vxleng.org

Pre-modules C++: the TU is the unit of compilation

```
C++ mesh_ops.cpp

#include <vector>           // textual paste: ~50k lines land here
#include <cmath>
#include "geometry.h"      // ...and get re-parsed, in every TU
#include "material.h"

double surface_area(const geometry::Mesh& m) {
    double a = 0.0;
    for (const auto& f : m.faces) a += area(f);
    return a;
}
```

```
BUILD Makefile

# the dependency graph is files → files
mesh_ops.o: mesh_ops.cpp geometry.h material.h
            c++ -c mesh_ops.cpp    # ~120k lines after preprocessing

render.o: render.cpp geometry.h material.h
            c++ -c render.cpp      # the same ~120k lines, again

app: mesh_ops.o render.o ...
```

- ▶ The TU is the unit of parsing, type-checking, codegen – **and scheduling**: parallelism is `make -j` across TUs, serial inside each
- ▶ Headers are textual inclusion: every TU re-parses the same standard-library lines
- ▶ One fat TU sets the wall clock; the scheduler cannot split it

Modules: parsed once – and a new scheduling edge

C++ geometry.ixx – exports a type

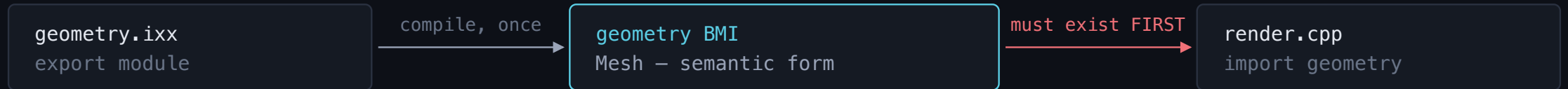
```
export module geometry;

export struct Mesh {
    std::vector<Face> faces;
    BoundingBox bounds() const;
};
```

C++ render.cpp – needs that type

```
import geometry;           // semantic, parsed once ✓

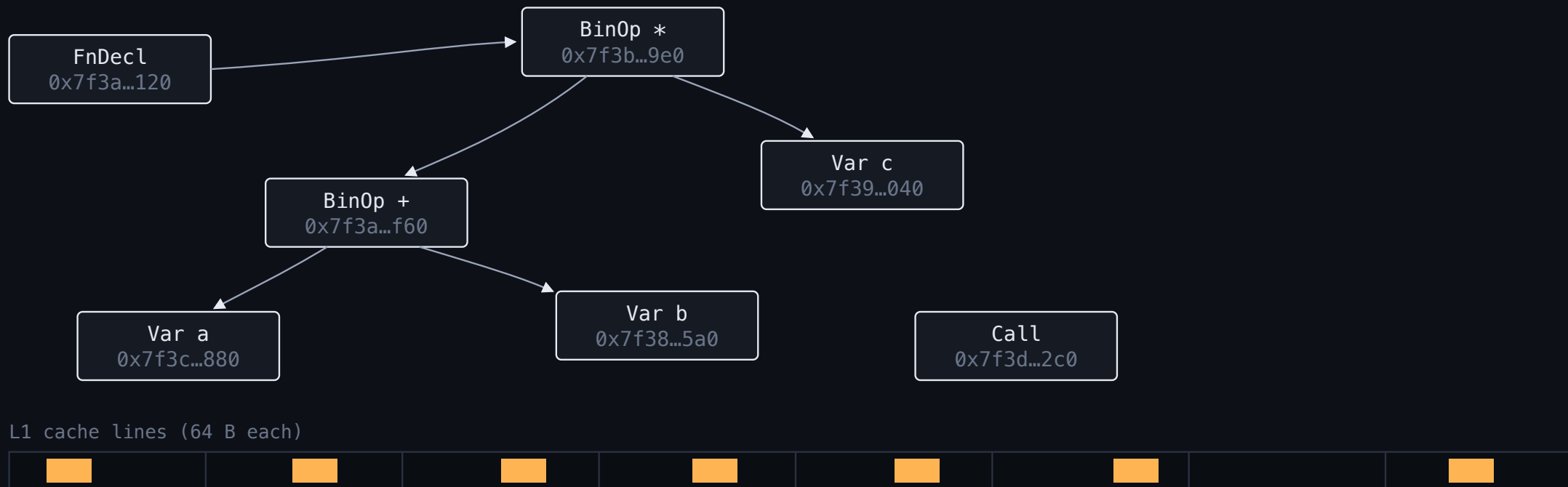
double lit_area(const Mesh& m) {
    return ...;             // but: geometry must be COMPILED
                             // before this TU can even start
}
```



The red edge is the new cost: a type dependency became a build-order dependency.

- ▶ Modules end the re-parsing – the importer reads a compiled, semantic artifact: the **BMI** (binary module interface – Clang's `.pcm`, MSVC's `.ifc`). Real progress
- ✗ And cross-TU type lookup now **serializes the build**: importers wait on exporters, and long import chains become the critical path
- ✗ The unit of scheduling is still the TU

An AST is a cache-miss generator



Seven nodes of `(a + b) * c` in a real allocator: seven cache lines touched, five dependent loads – each `->` stalls on the one before it. The prefetcher cannot help; the addresses are wherever `new` put them.

- Deeply nested trees $\Rightarrow$ pointer chasing on every walk, every pass, every query
- Node identity = allocation address $\Rightarrow$ meaningless across threads, useless on disk

Where a frontend reaches for global state

THE QUESTION

“what does this name mean?”

“have I seen `vector<int>` before?”

“what is `Mesh`, over in `geometry`?”

“did anyone already instantiate this template?”

THE SHARED STATE

symbol tables, written during resolution

the type interner – identity handed out in arrival order

another TU's tables

the shared instantiation cache

WITH THREADS

a lock

a lock, and a scheduling-dependent answer

a build-order edge

a lock

Every row is a rendezvous: a place where independent work has to stop and agree. Locks **prevent parallel compilation**.

The worst row is the second: identity by arrival order means different thread interleavings produce different IDs – and different output. That is untestable by construction.

Your linker already knows the trick

Two TUs, compiled on different machines on different days, agree on this symbol – because a mangled name is a **pure function of the fully qualified name and signature**. No TU ever asked another what it called things.

```
$ nm mesh_ops.o | grep area
_ZN8geometry4areaERKNS_4FaceE # derived from the name
$ nm render.o | grep area
_ZN8geometry4areaERKNS_4FaceE # same answer, independently
```

ENTITY	IDENTITY, COMPUTED IN ANY TU
<code>geometry::Mesh</code>	<code>hash("geometry") · hash("Mesh")</code>
<code>geometry::area(const Face&)</code>	<code>hash("geometry") · hash(def-path "area")</code>
<code>Vec<i32></code>	Vec's identity + digest of the arguments' identities

Make it **fixed-width** – 256 bits, a POD – and derived identity works *during compilation*: every phase, every thread, agreement without communication.

That struct is the GID. Part I builds it bit by bit.

Then the unit of parallelism can be a function

	TRANSLATION UNIT	FUNCTION
count, on the measurement corpus	1,000	16,000
shape	lumpy – one fat TU sets the tail	small and uniform – work-stealing keeps 48 cores fed
waits on	build-order edges: importers stall on exporters	nothing in flight – names resolve to computed GIDs against a frozen registry
granularity of failure	the whole TU	one body declines, falls back

A function needs two things to compile: its own body, and the *identities* of what it names. Once identity is computable and the shared facts are frozen, no function waits on another – sixteen thousand independent tasks instead of a thousand lumpy ones.

THE VEHICLE

Vx: one language, every core

A heterogeneous systems language. The thesis:

*"Heterogeneity belongs in the type system, not in the runtime. A host CPU dereferencing an NPU pointer should be a **type error**, not a segfault."*

- ▶ You *declare* the machine – memories, capacities, bandwidths, transfer costs – the compiler routes, admits, and rejects before anything runs
- ▶ Placement is type state: `Ref<T, Memory::NPU_HBM>`, `transfer()`, `spawn on(...)`

Frontend	~76k lines of Rust, no query engine, no locks
Backends	MLIR → LLVM 22 · NVPTX · Apple ANE · distributed
CPU / GPU	AOT or JIT; vendor kernels routed by <code>kind=</code>
Checked	address-space typing · borrow ck · linear types · capacity admission · topology reachability

Why the frontend matters: all of that checking is per-function semantic work. A language that proves placement needs a frontend that scales.

Where a value lives is part of its type

```
VX placement, typed

Memory GPU_HBM {
  within: Memory::CPU_DRAM, capacity: 40 GiB, bandwidth: 3 TB/s
}

fn main() -> i32 {
  let q_h = Tensor<f16>([8, 64]);           // lives in CPU_DRAM
  let q = transfer(q_h, Memory::GPU_HBM); // placement is typed
  // ... compute on the device ...
  let home = transfer(q, Memory::CPU_DRAM);
  print(home[0][0] as f32);
  return 0;
}
```

Leave out a `transfer` and the frontend answers with where the value lives, what the target can see, and the costed fix:

```
error[E6003]: 'host_data' lives in CPU_DRAM
  NPU[0] sees only [NPU_HBM]
  help: insert an explicit transfer to NPU_HBM
```

The declared `Memory` block is what makes both `transfer` calls checkable: the compiler knows what exists, what fits, and what can reach what – before anything runs.

Thursday's keynote makes this same case for C++ – that there is no word for *residency*, now that CXL and far memory have made tiers real – and answers with a library. This is that idea one level down: not an annotation the allocator honours, but a type the compiler holds you to.

THE IDEA, IN VX · A WORKING EXAMPLE

A function is a unit of compilation (almost)

VX

inc.vx

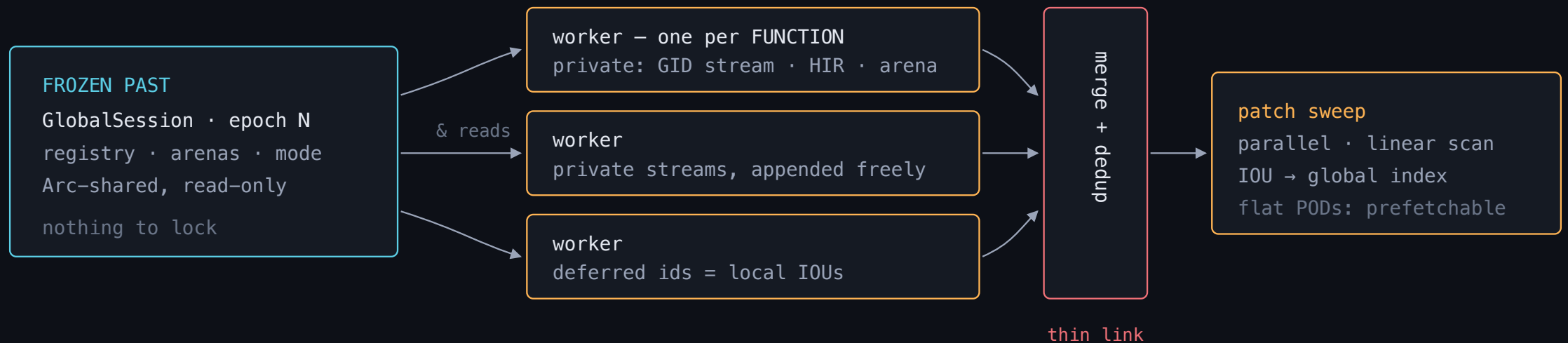
```
fn saturate(x: f32) -> f32 {
  if x > 1.0 { return 1.0; }
  return x;
}

// Ha = hash(Tensor, f32, [1024]) · Hb = H(u64): builtin
fn inc(a: &mut Tensor<f32, [1024]>, b: u64) -> i32 {
  for i in 0..1024 {
    a[i] = saturate(a[i] + b as f32); // a, b, i are locals;
  }                                   // saturate is a signature
  return 0;
}
```

WHAT <code>INC</code> NAMES	ITS IDENTITY	COMES FROM
Tensor<f32, [1024]>	hash(Tensor, f32, [1024])	computed right here
u64, f32	builtin hashes	known to every thread
a, b, i	locals	the body itself
<code>saturate</code> 's signature	frozen registry	read by <code>&</code> , no lock

Nothing in `inc` waits on anything in flight: it reads `saturate`'s signature, never its body. It can be checked on any thread, in any order, and later lowered into a worker's private arrays.

Parallel compiler design

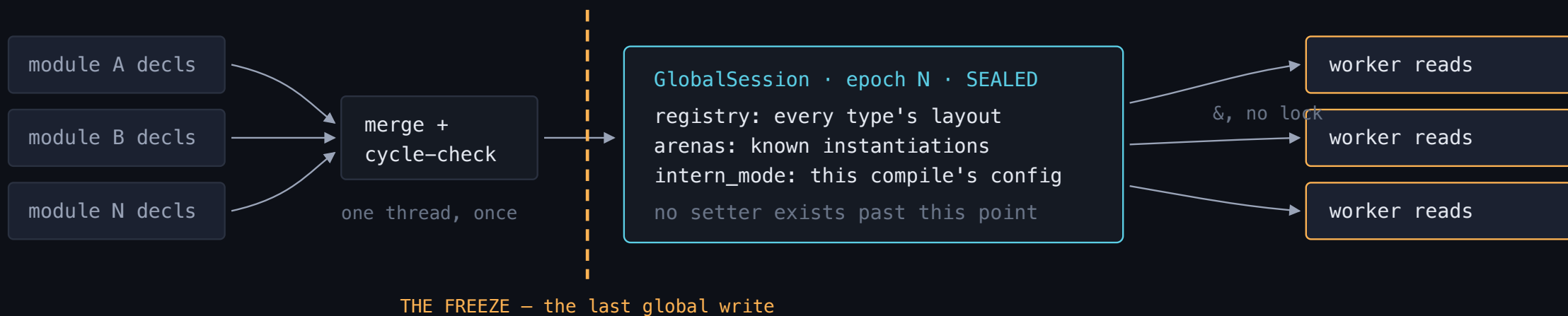


identity is computable: nominal types are content hashes (agreement without communication) · generics defer to the one barrier

Every worker reads a frozen past and writes a private present. The one question that would force workers to talk – what a type is called – is answered by hashing content, or deferred to a single reconciliation: it merges only the small argument-list arenas rather than complete program.

Parallelism is a property of your data. The rest of the talk walks this picture left to right: the identity (GIDs), the private arrays (flat HIR), the pipeline around them, and the tests that keep all of it true.

Frozen Past



Everything left of the line happens before the first worker exists. Everything right of it is read-only.

- ▶ `registry_freeze` merges every module's declarations into one `ImmutableGlobalRegistry`, cycle-checks it, and wraps it in an `Arc` – from here on, all sharing is by `&`
- ▶ “Sealed” is enforced by types: the builder is consuming (`in_intern_mode(self) → Self`), so a session is settled before it can be shared at all
- ▶ The C++ analogy: a whole-program binary module interface (BMI) – the compile's facts become an immutable artifact every task takes by `const&`

The freeze is a language decision

WHAT VX DOES NOT ALLOW

- ▶ **No inferred signatures** – every parameter and return type is explicit: checking a body needs signatures, never another body's check
- ▶ **No by-value recursive types** – recursion must box; every layout is computable at the freeze, and the cycle check rejects the rest
- ▶ **No specialization after the fact** – a generic's meaning closes at its definition: `Vec<i32>` = base + arguments, nothing else
- ▶ **No implicit conversions** – every conversion is an explicit `as`; assignment requires identical types. Literals get their type from context instead:

```
let v: i64 = if c > 0 { 7 } else { 9 };
// 7 and 9 are born i64
```

- ▶ **No user-defined assignment** – copy and drop are properties of the type, carried as GID flags (POD · needs-drop), settled at the freeze
- ▶ **No textual macros** – declarative macros expand in their own early phase, before names resolve
- ▶ **No comptime that mints declarations** – `if comptime` prunes and proves before body-checking commits

WHAT PREVENTS THIS IN C++

- ✗ The preprocessor – a header means something different per TU, per inclusion order
- ✗ `template<> struct hash<MyType>` – specialization after use retroactively rewrites an already-seen template
- ✗ Point-of-instantiation lookup + ADL – the same template, a different meaning per call site
- ✗ `auto f()` deduced across functions – checking a caller needs the callee's *body*
- ✗ Open overload sets – any header can grow the candidates for an already-written call
- ✗ Implicit conversions – which overload wins depends on every converting constructor and `operator T()` in scope, another set any header can grow
- ✗ `operator=` and special members – synthesized on first use, inside whichever body needs them first: checking a body *mints a new function*

One global fact can still be born inside a body: a **new generic instantiation**. That single exception gets the deferred-identity machinery of Part III – everything else was designed out of the language.

PART I OF IV

IDENTITY

256 bits, computed identically by every thread.

The GID: identity as a pure function of content

w0	module-path hash · FNV-1a-64 of "geometry::mesh"	← identity is
w1	symbol hash · FNV-1a-64 of the kind-tagged def path	← (w0, w1)
w2	payload – the borrow checker's facts, inline (next slides)	
w3	flags · visibility · POD · drop · inline attrs · DEFERRED · digest/arena selectors	

[u64; 4] · 32 bytes · #[repr(C)] · Pod – one cache line holds two

Identity is the composite 128-bit `(w0, w1)` – minted by hashing what the thing *is*. Every thread computes the same 256 bits independently: no table, no counter, no rendezvous, and nothing for scheduling to perturb. Collisions are rejected deterministically when the registry freezes.

The hash is the project's own pinned FNV-1a: a standard library may change its hash between releases, and a GID must reproduce byte-for-byte across builds, machines, and years.

Word 2 carries the borrow checker's facts

VX

what a signature has to remember

```
fn first(v: &Vec<f32>) -> &f32 {
    return &v[0];           // the result borrows from v
}

// at every call site:
let r = first(&x);          // r borrows x – but how does the
                           // checker know, without reading
                           // first's body again?
```

FACT THE CHECKER NEEDS

WHERE IT LIVES IN WORD 2

`v` is a reference, born at
scope depth 1

param-1 slot: region = 1 (12 bits) +
variance (3 bits)

the return value borrows from
`v`

return slot: provenance = “param 1”
(3 bits)

...and up to 3 more
parameters

four 16-bit slots fill the 64-bit word

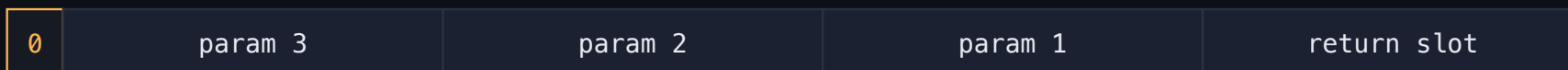
At the call site, “does `r` borrow `x`?” is answered by
reading bits from a GID that is **already in a register**. No
table, no AST walk, no lock.

Measured on a 177-crate Rust corpus: **99.7%** of reference-taking functions fit
these four slots.

When the facts don't fit: the escape hatch

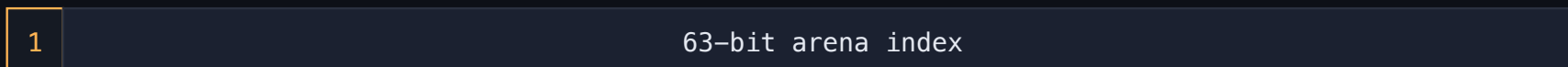
A signature with dozens of reference parameters cannot pack into 64 bits. Bit 63 picks the representation: the facts inline (the 99% case above), or an index into a side table.

bit 63 = 0



fast path – 99%: lifetimes/variance inline, [region:12 | variance:3 | r:1] per 16-bit slot, in registers

bit 63 = 1



slow path – <1%: index into a side table holding the full metadata

the escape-hatch bit: the 99% case never allocates; the 1% case never constrains the format

The dispatch this buys: `enum LifetimeSignature { FastPath(u64), SlowPath(&UnboundedFunctionMetadata) }` – the common case is a register-resident bitmask; the outlier is one predictable branch to a frozen arena.

Word 3: a few bits stand in for whole analyses

Dead code elimination.

```
fn helper(x: i64) -> i64 { ... } // no callers anywhere
```

Sweep the flat streams once, counting references to each GID. For a function with zero references:

- ▶ visibility bits (w3, bits 60-63) say **Private** ⇒ **delete it** – nobody outside this compilation can even name it. Decided from 4 bits on the ID
- ▶ bits say **FullyPublic** ⇒ keep it – an importer may link it later

One pass through a translation unit removes several redundancies – every check is a bit test on an identifier already in hand.

Drop and copy elimination.

```
struct Point { x: f32, y: f32 } // plain data
```

- ▶ **IS_POD** set (bit 44) ⇒ every copy lowers to **memcpy** – no copy calls emitted
- ▶ **NEEDS_DROP** clear (bit 45) ⇒ no destructor call at scope end – the cleanup code is never generated at all
- ▶ same trick for **#[inline]**, **#[cold]**, **#[must_use]** (bits 52-55): call-site decisions without loading the callee

Make identity a POD, and serialization is free

RUST – THE VX COMPILER

src/gid.rs

```

/// The 256-bit Global Identifier
#[derive(Clone, Copy, PartialEq, Eq, Hash,
    Pod, Zeroable)] // bytemuck: cast is legal
#[repr(C)]
pub struct TypeId {
    pub words: [u64; 4],
}

// zero-copy metadata files: length prefix + raw records
let ids: &[TypeId] = &worker.local_type_stream;
let bytes: &[u8] = bytemuck::cast_slice(ids);

```

C++ – THE SAME PATTERN

your codebase

```

struct GlobalId {
    std::array<std::uint64_t, 4> w;
    friend bool operator==(const GlobalId&,
                           const GlobalId&) = default;
};
static_assert(std::is_trivially_copyable_v<GlobalId>);
static_assert(sizeof(GlobalId) == 32); // no padding, ever

// zero-copy both directions
std::span<const GlobalId> ids{worker.types};
auto bytes = std::as_bytes(ids);

```

- ▶ Trivially copyable ⇒ `memcpy`-shaped serialization, mmap-able caches, hashing four `u64` s
- ▶ The `static_assert` s guard an on-disk format: a padded or resized ID silently breaks every file already written

PART II OF IV

THE FLAT PROGRAM

A function body as two contiguous arrays.

The program becomes two (flat) arrays

RUST – THE VX COMPILER

src/bytecode.rs

```
/// Flat-array bytecode replacing the AST
#[repr(C)]
#[derive(Clone, Copy)]
pub struct HirInstruction {
    pub opcode: Opcode,      // #[repr(u32)] · from_u32 is checked:
    pub operand1: Register,  // corrupt streams are rejected
    pub operand2: Register,  // u32 = index of an earlier inst
    pub type_idx: TypeIdx,   // u32 → the parallel GID stream
    pub imm: u64,           // inline – no constant pool
}

// the two arrays – one pair per worker (src/session.rs)
pub local_hir_stream: Vec<HirInstruction>, // the program
pub local_type_stream: Vec<TypeId>,       // its types
```

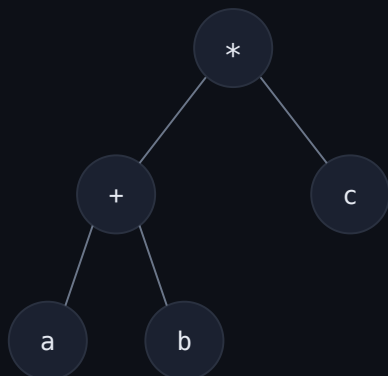
Every instruction is **24 bytes**, trivially copyable – a measured decision: at 16 B you fit four per cache line but need a side-band constant pool, and the pool's indirection cost more than the width. Inline `imm`, two per line, zero extra loads.

Types live in their own stream: an instruction sweep that ignores types never needs them.

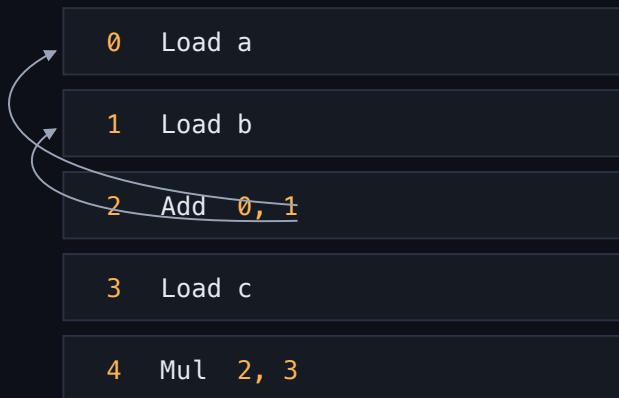
SSA-by-position: the index **is** the register

After a body type-checks, a flattening pass walks each expression bottom-up and appends one instruction per node. The result lands in **SSA form** and a value's name is simply the index where it landed.

```
fn f(a: f32, b: f32, c: f32) -> f32 { return (a + b) * c; } // the body being lowered ↓
```



(a + b) * c



Bottom-up emission: children are appended before their parent, so operands always point *backward*. A debug pass checks that every operand refers to an earlier index.

RUST – THE VX COMPILER

src/hir/flatten.rs · abridged

```
fn emit_typed(&mut self, op: Opcode, o1: Register, o2: Register,
              ty: LoweredTy, imm: u64) -> Val {
    let type_idx = TypeIdx(self.types.len() as u32);
    self.types.push(ty.gid());
    let reg = Register(self.code.len() as u32);
    // the index IS the register – no dest field
    self.code.push(HirInstruction::new(op, o1, o2,
                                       type_idx, imm));

    Val { reg, ty }
}
```

Every pointer becomes an index

TREE CONCEPT	FLAT ENCODING	WHY IT WORKS
child pointer	<code>operand = earlier index (u32)</code>	bottom-up emission guarantees “earlier”
type pointer	<code>type_idx → GID stream</code>	types cluster in their own contiguous array
nested block / scope	numbered basic block in <code>imm</code>	<code>CondBr.imm = then (else << 32)</code>
n-ary argument list	run of <code>Arg insts + count</code> in <code>Call.imm</code>	variable arity without variable-size nodes
5-operand intrinsic	<code>imm = o v << 16 scale << 32</code>	two operand fields are enough if <code>imm</code> is real

And the inverse of a classic optimization: clang `alloca`s every local and lets `mem2reg` promote what is never addressed. *Vx registers* every local and **demotes on address-taken** – escape analysis runs first, so the common local never touches memory at all.

Where the AST lives, and where it dies

- ▶ Parsing still builds a real tree – *“human code is inherently nested and tree-like.”* (The **only** phase where pointers exist)
- ▶ Name resolution writes GIDs *onto* the tree; flattening then **harvests** instead of recomputing
- ▶ Per function, lowering is **atomic**: a complete flat body, or an empty stream.
- ▶ After lowering, nothing reads the tree again – *“it is dead to the compiler”*; it is freed at teardown (static analysis tools can still hook up and use it)

PART III OF IV

THE PIPELINE

Freeze, fan out, reconcile once, patch.

The shape of a compile



17 timed phases · one worker per function – 16,000 on the 1,000-module corpus

every parallel site also has a plain serial form – the baseline compiler runs with no rayon in it

Serial points in red: the registry freeze (whole-program cycle check), `env_build`, the reconciliation barrier (where the workers' arenas merge and deduplicate), the epoch-2 registry copy that lowering reads its monomorphs from, and writing functions back to their corresponding modules.

What happens where

PHASE	RUNS PER	WHAT HAPPENS	-J 1 → -J 48 · MS
parse	file, in import waves	source to AST; <code>vx</code> <code>-j</code> follows imports one wave at a time	280 → 13
macro_expand	module	the macro table is collected once, then every module expands on its own	61 → 6
name_resolution	module	top-level names get GIDs, then every type reference gets its GID attached	37 → 2
freeze · env	serial	the frozen registry: layouts, signatures, the method table, the by-value cycle check; then the checker's read-only view of every declaration (generic bodies kept for instantiation)	133 → 139
type_check	function	check one body; generic calls become mangled monomorphs, checked here too; the signature's generic types get deferred GIDs; a capacity summary goes out	227 → 19
dedup_barrier	serial	every worker's arena into one, deduplicated by argument list; a local→global map per worker	12 → 13
stream_extract · simd_patch	stream	each deferred word 2 becomes its global index and the bit clears	1 → 1
codegen: route · epoch 2	serial	monomorphs to the module that owns the generic, sorted, deduplicated by name; the registry plus one signature per survivor	142 → 122 †
codegen: lower · emit	function	body to flat HIR against epoch 2 (a call stores its callee's GID), then flat HIR to MLIR text; numbering is a function of position, so any thread order gives the same bytes. One serial step between the two builds the callee map	930 → 64
reassembly	serial	every function's text written back in module order: the figure's write-back	†

Red is the serial spine; epoch 2 and write-back are timed together (†). Medians of 10 runs on the 1,000-module corpus, c5.metal; with teardown, the rows add up to the ladder's 1.96 s → 0.45 s.

A frozen past, a thread-local present

RUST — THE VX COMPILER

src/session.rs · abridged

```
/// The frozen past of the compilation.
pub struct GlobalSession {
    pub epoch: u64,
    pub intern_mode: InternMode, // settled BEFORE sharing
    pub registry: Arc<ImmutableGlobalRegistry>,
    pub slow_path_arena: Arc<Vec<Metadata>>,
    pub generics_arena: Arc<Vec<TypeId>>,
}

/// The mutable present of ONE worker.
pub struct LocalWorkerState {
    pub global: Arc<GlobalSession>,
    // completely lock-free, thread-local mutation
    pub local_type_stream: Vec<TypeId>,
    pub local_hir_stream: Vec<HirInstruction>,
    pub local_generics_arena: Vec<TypeId>, // + 6 more
}
```

The arenas are homogeneous and strictly typed – the design doc names cache-line density and hardware prefetching as the reasons.

The session's setter is *consuming* (`in_intern_mode(self) → Self`): the type system makes “settled before shared” a compile-time fact.

The deferred bit: “this name is temporary”

Mid-check, a worker meets `Vec<i32>` – a name *born during checking*, so the frozen registry has never heard of it. The worker needs an identity right now, without talking to anyone.

- ▶ From Part I: bit 63 of word 2 set $\Rightarrow$ word 2 holds an **index into a side table**
- ▶ **DEFERRED** set (a word-3 flag) $\Rightarrow$ the index points into **this worker's private arena** – a temporary name, valid only here
- ▶ **DEFERRED** clear (minted at the freeze, or patched after the barrier) $\Rightarrow$ the index points into the **frozen global arena** – a final name, valid everywhere

So a worker can mint a usable identity instantly and keep checking. Making the name final is postponed – that's the slides that follow.

worker 7's private side table – one flat Vec:

```
local_generics_arena    [ H(i32) ][ H(Str) ][ H(f32) ]
                        start=0    start=1
local_generics_offsets  #0 = (start 0, len 1)  #1 = (start 1, len 2)
```

#0 = Vec<i32>'s argument list · #1 = Map<Str, f32>'s

the GID minted for Vec<i32>:

```
word 2 = 0           (its entry number in the table above)
word 3 = DEFERRED    ("this index is mine – translate later")
```

`start = arena.len()` marks where the next argument list will begin; the list is appended; `(start, len)` is recorded so entry `#k` can be sliced back out as `arena[start .. start+len]`.

Each `H(·)` is the argument's full GID, hashed from its **fully qualified name** – and the minted GID keeps `Vec`'s own module + symbol words (`w0`, `w1`). So `std::Vec<i32>` and `my::Vec<i32>` can never collide.

Minting `Vec<i32>` without asking anyone

RUST – THE VX COMPILER

src/pipeline.rs · abridged

```
fn mint_deferred_generic(worker, base, args) -> TypeId {  
    let start = worker.local_generics_arena.len(); // next free slot  
    let len = args.len();  
    worker.local_generics_arena.extend(args); // no lock  
    let index = worker.local_generics_offsets.len() as u64;  
    worker.local_generics_offsets.push((start, len));  
  
    let mut id = base; // inherit module + symbol words  
    id.set_arena_index(index, Arena::Generics, Scope::Local);  
    id // carries LOCAL_DEFERRED_BIT – an IOU, redeemed at the barrier  
}
```

A generic instantiation needs global agreement on its name – and agreement is a lock.

So agree later. Park the argument GIDs in **your own arena**, put *your local index* in word 2, set the **deferred bit**.

- ▶ Zero locks, zero atomics, zero waits – a `Vec::extend`
- ▶ The deferred bit routes every later read to the right memory: worker-local vs frozen-global, one predictable branch

A deferred identity is a relocation

What just happened, step by step:

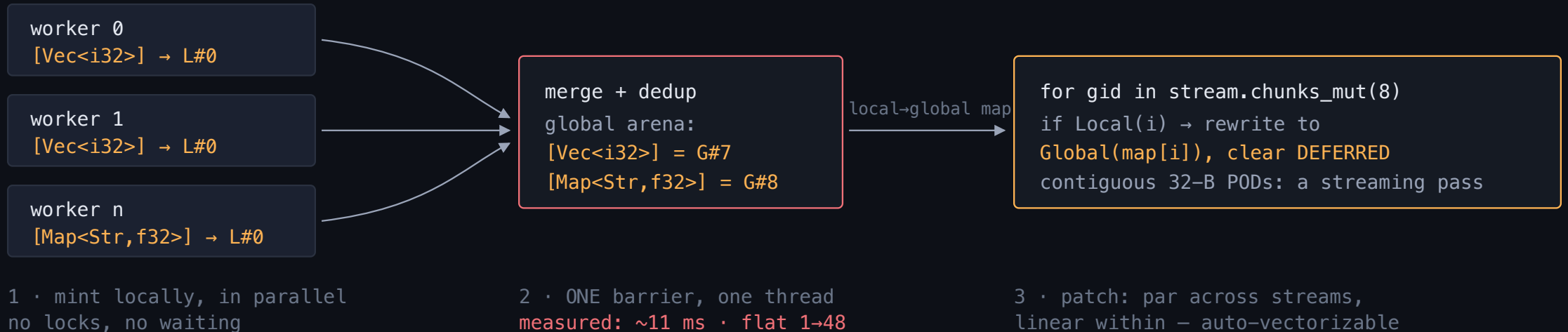
- ▶ Worker 7 meets `Vec<i32>`. The global name isn't decided yet – deciding now would need a meeting
- ▶ So it records the arguments in its own arena, slot 0, and stamps the GID `Local#0` · `DEFERRED` – a placeholder with the real content behind it
- ▶ Worker 12 meets `Vec<i32>` too and mints its own `Local#0` · `DEFERRED`. Neither waits
- ▶ At the barrier, both slots hold the same argument list `[H(i32)]` → one global slot `G#7`; each worker gets its map `L0 → G7`
- ▶ The patch pass rewrites every placeholder to `G#7` and clears the bit

You already ship this pattern. An object file names a symbol it cannot place, and the linker fills the address in one pass:

```
$ objdump -r mesh_ops.o
RELOCATION RECORDS FOR [.text]:
OFFSET      TYPE          VALUE
0000001a R_X86_64_PLT32  _ZN8geometry4areaE...
# "someone named area goes here" – patched later
```

A deferred GID says “the generic `[Vec, i32]` goes here.” The barrier assigns the final index, and **the patch pass is the linker.**

One barrier, then a linear sweep



The graph fixup became a linear scan: because identities live in flat arrays of PODs, “rewrite every deferred reference” is a chunked sweep the prefetcher can see coming. A debug hook then asserts **no deferred bit survives anywhere**.

Minting `Vec<i32>::len()`: a name is enough

RUST — THE VX COMPILER

src/hir/check/calls.rs · src/pipeline.rs · abridged

```
// worker 7, checking a caller: let n = v.len(); // v: Vec<i32>
let (generic, mapping) = resolve_method_in_impls(Vec<i32>, "len"); // T = i32
let mut len = instantiate_function(&generic, &mapping); // clone, substitute
len.name = "Vec$i32$len$i32"; // receiver.mangle() $ method $ bindings
if !worker.monomorphs.has(&len.name) {
    check_function(&mut len); // its body, right here, no lock
    worker.monomorphs.push(len);
}
*expr = FunctionCall("Vec$i32$len$i32", [&v]); // the AST rewritten in place

// after the join, once:
route_monomorphs(..); // to the generic's module; sort by name; dedup by name
epoch_2_registry(..); // the frozen tables + one signature per survivor:
// gid = hash(module 0, "Vec$i32$len$i32") - a pure function of the name
```

A type needed the barrier because its identity carries an argument list. A function's identity is the hash of its mangled name.

- ▶ Two workers that never meet mint the **same name**, so nothing is deferred and nothing is patched
- ▶ The only reconciliation is dropping duplicates by name after a sort, so the survivor does not depend on who got there first
- ▶ Every body is lowered against epoch 2, so a caller's `Call` stores that hash and the emitter's callee map computes the same one
- ✗ The bill: each calling function checks the instantiated body again. Checked once per caller, lowered and emitted once

```
func.func @Vec$i32$len$i32(%arg0: !llvm.ptr {llvm.readonly})
-> i32
```

PART IV OF IV

EVALUATION

CI + scaling tests + verifiers.

Zero locks – because CI says so

CI .github/workflows/ci.yml · abridged

```
- name: Check for locking primitives
  run: |
    if grep -rE '(Mutex|RwLock|Condvar|OnceLock|OnceCell
                |Semaphore|DashMap|parking_lot|spin::
                |std::thread::park)' src/; then
      echo "Locking primitives are not allowed!"
      exit 1
    fi

- name: Check for process-global atomics
  run: |
    if grep -rE '(std::sync::atomic|Atomic(U8|U64|...))' src/ \
      | grep -v 'vx-lint: allow-atomic'; then
      exit 1
    fi
```

Current state of `src/`:

locks / condvars / once-cells	0
exempted atomics	4 lines, one file
...of which decide any output	0

Each exemption carries a written justification in-file (the import, a log gate, and the phase timers – accumulated *from inside* the regions being timed, so there is no per-worker place to put them).

grep: It's crude, fast, but brittle.

*"No global mutable state remains; the lint passes **by construction**."*

commit 83f3fe71

Assert the architecture at every boundary

RUST – THE VX COMPILER

src/parallel_architecture_verifier.rs

```
// debug builds only – zero release cost
pub fn verify_phase_3_isolation(workers, global) {
    for worker in workers {
        assert_eq!(Arc::as_ptr(&worker.global),
                  Arc::as_ptr(global), // aliasing proof:
                  "worker diverged from the frozen session");
        for gid in &worker.local_type_stream {
            if let Index { index, arena, scope: Local } = ... {
                assert!(index < arena_len(arena)); // bounds
            }
        }
    }
}
```

Eight hooks at the phase boundaries the verifier owns:

- ▶ Every worker points at *the same* frozen session – shared-mutability drift caught by pointer equality
- ▶ Every deferred index is in bounds for *its own* arena
- ▶ After dedup: no duplicate metadata survives
- ▶ After the patch: **no deferred bit exists anywhere** – “absolute identity” or a crash
- ▶ After serialization: byte length is exactly $8 + n \cdot 32$

Invariants you can state are invariants you can assert. Asserts are the cheapest verification layer you will ever ship.

Byte-identical at any thread count

RUST – THE VX TEST SUITE

src/pipeline.rs tests · shape

```
let mlir_1t = pipeline_at_threads(1, corpus);
let mlir_4t = pipeline_at_threads(4, corpus);
let mlir_seq = pipeline_sequential(corpus); // rayon OFF the path

assert_eq!(mlir_1t, mlir_4t); // byte for byte, in order
assert_eq!(mlir_1t, mlir_seq); // baseline = same compiler
```

Compared byte-for-byte: every symbol, every SSA number, every string-global index – *“precisely the things a parallel emitter could number by arrival order if it were written carelessly.”*

Why it can hold at all: identity is content-hashed and numbering is positional. There is *nothing for scheduling to perturb*. Determinism gets designed in, then pinned with a test.

Checked at the size the speed-up was measured on: 1,000 modules, 16,000 functions, 36.2 MB of MLIR – identical bytes at 1 thread, at 4, and with rayon off the path. In CI on every commit.

Baseline discipline: `Sequential` takes rayon off the path entirely – plain `iter()` where the parallel form uses `par_iter()`. Otherwise the machinery's cost appears on both sides of every ratio and cancels out. Measured this way, the machinery costs 1.00× when unused.

Keep the old compiler as your oracle

The AST path was **demoted to reference implementation**.

- ▶ 130 differential tests: flat MLIR and AST MLIR through the *same* LLVM lowering, the *same* JIT – must produce the same result
- ▶ A function the flat path cannot handle yet is **declined**: the lowering returns an empty stream and the AST path compiles that function. The compiler never guesses
- ▶ Corpus sweeps: every program the flat path accepts is byte-compared against the oracle

ThreadSanitizer – with a positive control

The whole compiler, sanitized end to end:

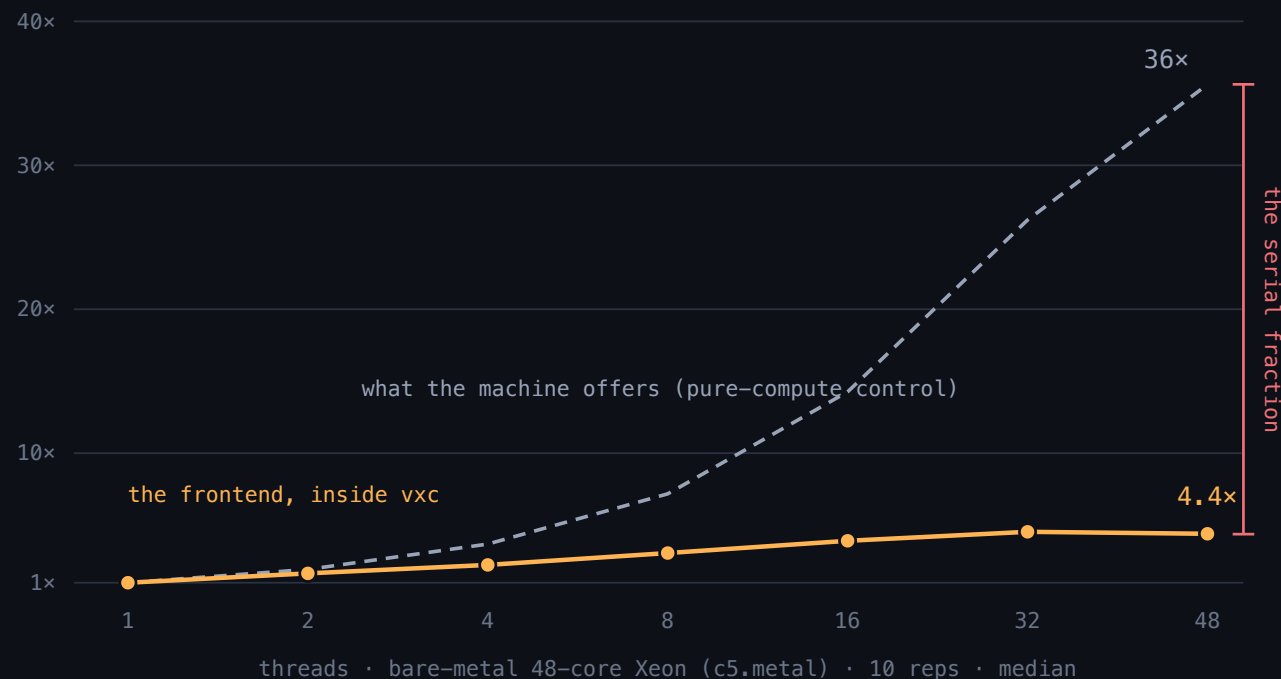
```
export RUSTFLAGS="-Zsanitizer=thread -C link-arg=-fuse-ld=lld"
export TSAN_OPTIONS="halt_on_error=0 suppressions=tsan.supp"
cargo +nightly build -Zbuild-std --target x86_64-unknown-linux-gnu --release
setarch -R cargo +nightly test -Zbuild-std --target ... --release --lib
# setarch -R: fixed address layout, so the shadow map is reliable
```

Before believing “0 races,” the harness compiles a *deliberate race* with the same flags and **requires** TSan to catch it:

```
# purpose is to race, and require a report
if grep -q "data race" "$CTL_LOG"; then
    echo "detector works"
else
    echo "FATAL: control did NOT report a race"
fi
```

Result: **zero data races** at 1, 8, 32, and 48 threads, across 430 instrumented tests.

Evaluation: 1,000 modules, 48 cores, through vxc



1,000 modules, 16,000 functions, `vxc -j N --emit-mlir` on the shipping binary, 10 reps, median. The line is the frontend, parse to MLIR text. The dashed line is the theoretical limit.

THE FRONTEND · PARSE → MLIR TEXT · -J 1 → -J 48

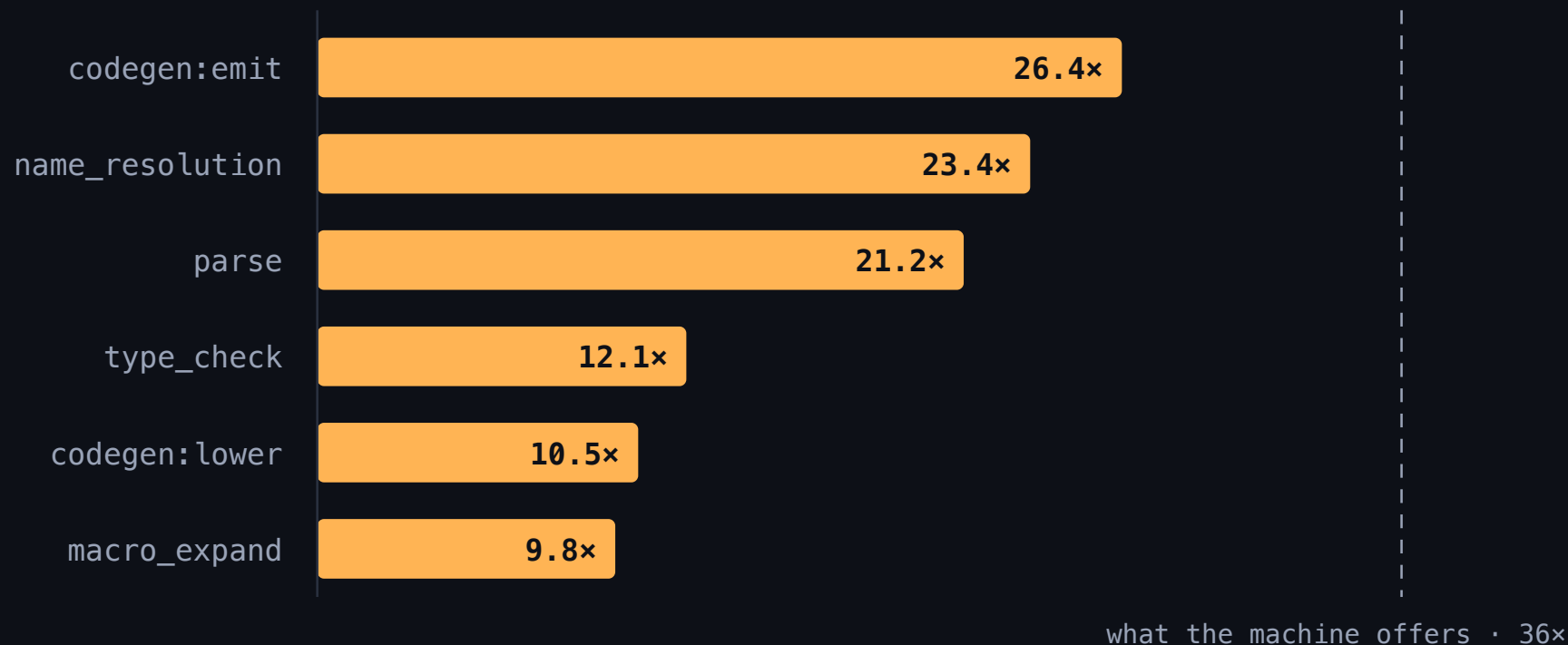
1.96 s → 0.44 s · 4.4x

WHAT THE MACHINE OFFERS

36x

The frontend's **serial fraction** caps it near 5x on any number of cores; the next slide shows what scales and what does not. The whole compile is 6.8 → 5.4 s.

Inside the 4.4x: the parallel work does scale



Speedup of each parallel phase at 48 threads, `-j 1` baseline.

Emit and parse scale best because small per-function working sets and nothing shared. Type check and lowering read the frozen registry on every step, and a shared read-only table still costs cache traffic at 48 threads.

Why not just make `-j`?

Two dials. `make -j M` schedules *across* translation units; `vxc -j N` schedules *inside* one. The unit decides which dial matters, not the core count.

48 CORES · M PROCESSES × N THREADS = 48

THE DIAL THAT WORKS

THE OTHER DIAL

1,000 modules of 16 functions

 $M = 48, N = 1 \cdot 1.1 \text{ s}$ $M = 1, N = 48 \cdot 24.8 \text{ s}$

one 16,000-function file

 $N = 48 \cdot \text{frontend } 1.82 \text{ s} \rightarrow 0.77 \text{ s}$

M has one unit to schedule

Given both dials, the build system can choose the right mix based on file size and file count.

Six patterns, and where C++ says no

1 · FLATTEN

Graphs into arrays of trivially-copyable records; pointers into u32 indices. Passes become sweeps; serialization becomes memcpy.

2 · CONTENT-HASH IDENTITY

Identity derives from content alone. Every thread mints the same ID with zero coordination.

3 · DEFER IDENTITY

When a thread cannot know the final ID yet, it makes a temporary one with a promise bit set and keeps going. One cheap barrier settles them all and patches the references in a single pass.

4 · FREEZE THE SHARED

Anything shared is read-only: `shared_ptr<const T>`. Anything mutable stays on one thread. Finish setting it up before you share it, and let the compiler enforce that order: setters that take the object by value cannot be called once it is shared.

5 · BAN THE PRIMITIVES

CI greps for them; exemptions carry written reasons. When flagged, prefer deleting the shared state to making it lock-free.

6 · DISTRUST THE METER

Baselines with the machinery absent; a hardware-ceiling control; positive controls for every detector; measure the binary users run.

Not all of C++. The six are about data and discipline. A deduced `auto`, `constexpr` evaluation and template instantiation break, each needing a callee's body, and textual `#include` keeps parse serial per TU until modules.

The safety profiles might fit parallel compilation. P3081's type, bounds, initialization and lifetime profiles are each decidable from one function's body and the declarations it sees.

Where this stands

- ✓ Flat codegen is the shipping compiler's default backend; the oracle (the AST path) is still supported.
- ✓ Zero locks in `src/`; 4 exempted atomic lines; byte-identical output at any thread count
- ✓ The flat path compiles all but 12 of about 375 test programs; the 12 decline to the oracle. Swept and gated in CI
- ✓ `vxc -j N` runs the pipeline.

Next, in order:

- ▶ The backend round trip: the flat emitter writes MLIR text that the driver parses back and prints.
- ▶ The frontend's serial parts that can be further optimized.
- ▶ Growing the flat subset until the oracle (the AST path) retires

The bill for a closed world

A closed world cannot see past a call edge: summaries in the parallel phase, one fold after it, never a worker asking for another function's result mid-check.

- ✗ **Recursion** that touches a bounded space needs a `requires` clause when the depth cannot be proven.
- ✗ **Unimplemented language features** have to be designed in this shape before they land. An asynchronous `spawn` etc.

Tracking issue: Vx#444.

VX

on the host · fine

```
fn fact(n: i32) -> i32 {  
  if n <= 1 {  
    return 1;  
  }  
  return n * fact(n - 1);  
}
```

VX

in a bounded space · the lift, Vx#444

```
// refused (E6028) without the clause: depth, and residency, unbounded  
fn fact(n: i32) -> i32 requires n < 16 {  
  if n <= 1 {  
    return 1;  
  }  
  return n * fact(n - 1);  
}
```

A whole-program analysis that keeps the frontend parallel

RUST — THE VX COMPILER

src/hir/check/capacity_fold.rs · src/pipeline.rs · abridged

```

/// What ONE function's check exports: a few dozen bytes.
pub struct FnCapacitySummary {
    pub name: String,
    pub self_peak: BTreeMap<String, u64>, // space → peak bytes
    pub calls: Vec<CallEdge>, // live bytes at each call
}

pub struct CallEdge {
    callee: String, live: BTreeMap<String, u64>, span: Span,
}

// After the parallel phase, ONCE, reading summaries only:
let summaries: Vec<_> = check_results.iter()
    .flat_map(|c| c.capacity_summaries.iter().cloned()).collect();
fold_cross_call_capacity(&summaries, global_env, &mut diags);

// total(f, s) = max( self_peak(f, s),
//      max over calls c: live(f, c, s) + total(callee(c), s) )
// cycles collapsed first (Tarjan) · O(functions + calls)

```

The question that needs a call graph: **what is the peak memory along any call path?** A tile the caller still holds while the callee runs joins the callee's working set. A call graph is the one global structure the freeze exists to avoid.

The answer that keeps the freeze: each worker's check already walks its function's placements to find the peak. It keeps that number, records what is still resident at every call, and exports both. **One fold at the join after type check** composes them: sequential calls by `max`, a tile held across a call by `+`.

- The fold reads summaries only. No body, no AST, no type stream is touched after the fan-out
- A program that declares no capacity costs nothing; one that does costs integer arithmetic over a few dozen bytes per function
- It only ever adds refusals. What the per-function check admitted stays admitted in isolation

You ship this pattern already: `-fstack-usage` writes a per-unit summary and a link-time pass folds it over the call graph. Same recurrence, per memory space instead of one stack.

Out of memory, as a compile error

VX

a 4 MiB space, two 3 MiB tiles

```
Memory W {
  within: Memory::CPU_DRAM, capacity: 4 MiB, managed: explicit
}

fn g() -> i32 {
  let y = Tensor<f32, [1024, 768]>::uninit(); // 3 MiB
  let _sy = transfer(y, Memory::W);          // g peaks at 3 MiB: fits
  return 1;
}

fn f() -> i32 {
  let x = Tensor<f32, [1024, 768]>::uninit();
  let sx = transfer(x, Memory::W);           // f peaks at 3 MiB: fits
  let r = g();                               // sx is read below: live
  let _back = transfer(sx, Memory::CPU_DRAM);
  return r;
}
```

Each function fits by itself, so a per-function budget admits both. The fold sees the path:

Error[E6027]: the working set along call path 'f -> g' in memory space 'W' peaks at **6291456** bytes, over its 4194304 byte capacity: 'f' holds 3145728 bytes across its call, 'g' itself peaks at 3145728 bytes

THE SAME TWO TILES

TRUE PEAK VERDICT

f holds one, calls g , which places one	6 MiB	E6027, the path above
f() returns, then g() is called	3 MiB	admitted: max, not sum
deep(n) places one and recurses	$n \times 3$ MiB	E6028: depth unknown, refused

Row two is why you cannot fix this by adding the numbers up. Row three is the bill from the previous slide; the `requires` clause there is the planned lift (Vx#444).

No hardware, no run: an out-of-memory before anything executes, from the same parallel pass that checked every function, plus arithmetic.

CLOSING

**Trees are nice,
cache misses are not.**

Vx – one language, every core · questions welcome

appendix: keeping it parallel – the lint, one fix and its trap, a measurement story →

APPENDIX · IF TIME, OR FOR Q&A

KEEPING IT PARALLEL

Designing a parallel compiler is a project.
Keeping it parallel is a practice. Two stories behind the lint, and
one piece of prior art.

The rule behind the lint: no global registries

Any `static` / `thread_local` / `LazyLock` registry that one phase writes and another reads breaks the design: a fact recorded on parse-worker A is invisible to type-check-worker B. Per-compilation facts travel **on the program representation**, shared by `&`.

HISTORY 1 · THE REFLEX

```
LazyLock<RwLock<HashMap>>
```

Hardware-topology declarations in a global map; tests serialized by their own Mutex. Trips the lint *and* the design.

HISTORY 2 · THE STOPGAP

```
thread_local! RefCell
```

Passes the lint. Correct only while each compile is single-threaded – a declaration on one worker stays invisible to another. A trap, waiting.

LANDED · THE RULE

data carried on the AST → session

Registry deleted. Declarations flow like every other program fact. Nothing to lock because nothing is global.

The regression test: **64 threads** released by a barrier, each compiling a program that declares a topology with the *same name* and a *different* definition – each must see only its own. A registry of any flavor fails it; data-carried state can't.

Hidden locks in innocent code

- ▶ `println!` is a lock – the global stdout mutex, *inside the parallel region*
- ▶ A shared counter is a lock – numbering strings serialized the whole MLIR emit
- ✓ The fix: **prefix sums** – numbers come from position; nothing is shared

Zig: same flat data, different identity

	ZIG	VX
Program data	flat – struct-of-arrays nodes, <code>u32</code> indices, side table for wide nodes	the same idea, carried into the HIR and type streams
Parse → IR	parallel per file – “a pure function of that file’s contents”	parallel per file
Semantic analysis	thread-safe by design, and overlapped with codegen and linking since 0.15; upstream still analyses units one at a time	parallel per function – ~16,000 workers
Type identity	a 32-bit index <i>assigned</i> by a central InternPool	256 bits <i>computed</i> from content – there is no pool to ask
Concurrency strategy	make the meeting cheap: sharded pool, a mutex per shard, lock-free reads afterwards	remove the meeting: hashed names, plus one ~11 ms barrier for generics
Locks in the compiler	per-shard mutexes, by design	zero, enforced by CI
Shipping today	incremental compilation, self-hosted backends, years of production use	the pipeline runs in the shipping driver behind <code>vxc -j</code> ; no incremental compilation yet

Worth noticing, because both projects reached for it independently: Zig’s multi-threaded InternPool index puts **the thread id in the high bits** and a thread-local index in the low bits. A deferred GID sets a **DEFERRED** flag beside a worker-local arena index. Same instinct – write thread-locality into the identifier. Zig keeps that as the final name; Vx patches it to a global one at the barrier, which is what lets a GID mean the same thing in another module, on another machine, next year.

And the concession that matters: Zig’s `comptime` runs arbitrary code and generates types *during* compilation, so those names cannot always be hashed from a declaration path. A central pool is part of what that power costs. Hashed identity is available here because the language gave things up for it – the co-design argument, pointed the other way.